



UniAssist

Centralized AI Student Portal — MAJU

FYP-1 Evaluation & Viva Handbook

Everything to present, explain & defend the project — model, libraries, pipeline, demo script, expected questions & answers.

Project Team	Registration
Syed Ibrahim	SP23-BSCS-0178
Mubashir Riaz	SP23-BSCS-0208
Adnan Kamal	SP23-BSCS-0179
Abdul Rehman	SP23-BSCS-0181

Program: BS Computer Science | Supervisor: _____ | June 14, 2026

Inside this handbook

1. Project at a Glance — the 30-second pitch, problem & solution
2. FYP-1 Scope — what is built now vs what comes in FYP-2
3. System Architecture — the three-tier design & why
4. The AI Engine — model choices & why we picked them
5. What is RAG — explained simply (the heart of the project)
6. Technology Stack & Libraries — every package and why
7. End-to-End Pipeline — build-time & live query flow
8. Key Features — what each module does
9. How to Present — start-to-finish demo script
10. Possible Questions & Answers — 35+ likely questions
11. Anticipated Objections & confident responses
12. Cheat Sheet — numbers & one-liners to memorise

How to use this in the viva

Read sections **1, 4, 5 and 7** until you can say them in your own words — that is 80% of what examiners ask. Keep section **12 (cheat sheet)** open on your phone during the exhibition. Each team member should own the sections they demo (see the speaker split in section 9).

1. Project at a Glance

The 30-second pitch (memorise this)

“UniAssist is a centralized AI student portal for Muhammad Ali Jinnah University. Today, a MAJU student has to dig through the website, call offices, or visit in person for things like admissions info, fees, course details, or to report a problem. UniAssist puts all of that in one place: a smart chatbot that answers university questions in English *and* Roman Urdu — even by voice — and a full issue-tracking system that routes complaints to the right department. It has three kinds of users: students, department staff, and an admin, each with their own portal.”

The problem

MAJU's information and student services are **scattered**. Answers live across many web pages; getting help means physically visiting an office or sending an email that may go unanswered. There is no single, always-available, student-friendly point of contact — and nothing that understands the way Pakistani students actually type (Roman Urdu, mixed language, short SMS spellings).

Our solution

- **AI Chatbot** — answers MAJU questions grounded in the official website, in English / Roman Urdu / mixed, with optional voice input.
- **Issue Tracker** — students raise a ticket to a specific department (SFC, IT, HOD); staff resolve it; the student is notified by email + in-app bell.
- **Role-based portals** — separate, secured experiences for **Student**, **Staff**, and **Admin**.
- **Admin control** — manage users, staff, departments, view analytics, and even edit the chatbot's knowledge base live.

Who uses it

Role	What they can do
Student	Chat with the AI, raise & track issues, manage their profile
Staff	See only their department's issues, reply, change status, coordinate with teammates
Admin	Full oversight: users, staff, departments, analytics, knowledge base, audit logs

One-line definition (if you only say one sentence)

“UniAssist is a full-stack, AI-powered student portal for MAJU whose core is a Retrieval-Augmented chatbot running on a **fully local** LLM, backed by role-based Student / Staff / Admin web portals.”

2. FYP-1 Scope — Built Now vs FYP-2

This is important to say **up front** so the panel evaluates you on the right scope. In FYP-1 we delivered the **core platform, the complete AI chatbot engine, and the full role-based portals**. The remaining modules are intentionally scheduled for FYP-2.

Completed in FYP-1

Module	Status	What works today
AI Chatbot (RAG)	DONE	Answers MAJU questions from the website; voice input; English / Roman-Urdu; profanity filter + admin flagging
Issue Tracker	DONE	Submit to a department, attachments, status workflow, 20s live sync, email + bell notifications
Staff Portal	DONE	Dashboard, department inbox, issue detail, status & reply, multi-staff coordination
Admin Panel	DONE	Dashboard, users, staff, departments, query analytics, live vector-DB editor, audit/login/chat logs
Auth + RBAC	DONE	Register with MAJU email, OTP, password recovery, JWT, three roles
Profile (all roles)	DONE	Avatar, display name, change password

Planned for FYP-2 (say: “baad mein / next phase”)

Module	Status	Planned scope
Job Portal	FYP-2	AI helps search jobs & builds a profile from an uploaded CV
Scholarship Portal	FYP-2	Bot matches students to scholarships by academic profile
MAJU Events	FYP-2	Society & official events with filters
Unified Profile	FYP-2	General + job + scholarship profile in one view
Mobile App	FYP-2	Cross-platform React Native app with push notifications
Content Management	FYP-2	Admin CMS for events / jobs / scholarships

How to frame scope to the panel

“In FYP-1 our goal was to build the **foundation and the hardest technical piece — the AI**. So we completed the full chatbot pipeline, the issue-tracking system, and all three role-based portals end to end. The student-facing job, scholarship, and events modules, and the mobile app, are planned for FYP-2 because they sit *on top of* this foundation.” You’ll notice empty *Jobs / Scholarship / Events* pages already exist as placeholders — that is deliberate, so the navigation and routing are ready for FYP-2.

3. System Architecture

UniAssist is a **three-tier system** made of three independent services that talk over HTTP. Splitting them this way means each tier uses the best tool for its job and can be developed, scaled, and deployed separately.

FRONTEND (client/)	BACKEND (server/)	AI SERVICE (python/)
React 19 + Redux Toolkit Tailwind + Vite Student / Staff / Admin UI	Node.js + Express MongoDB + Mongoose Auth, APIs, RBAC, audit, notifications, file uploads	FastAPI + Uvicorn ChromaDB + Ollama RAG chatbot, moderation, language detect, Whisper STT

Data stores: **MongoDB** holds users, issues, chats, notifications, audit/login logs. **ChromaDB** (a vector database) holds the chatbot's knowledge — 4,763 text chunks scraped from the MAJU website.

How the tiers talk

- **Browser → Node:** the React app calls the Express API (JWT in the header) for all login, issue, admin, and notification actions.
- **Browser/Node → Python:** chat questions and voice notes go to the FastAPI service (/ask, /transcribe); the admin's knowledge-base editor calls Python's /chunks directly.
- **Python → Node (internal webhook):** when the bot detects abusive language it calls Node's /internal/flag-user with a shared secret, so the student is flagged and the admin is alerted.

Why a separate Python service?

The AI/ML ecosystem — embeddings, ChromaDB, Whisper, the Ollama client — is Python-first. Rather than force these into Node, we isolate them in a FastAPI microservice. Node stays a clean business-logic/API server; Python is the dedicated “AI brain.” They are loosely coupled over HTTP, so we could swap or scale either side without touching the other.

Security between services

Two internal endpoints are protected by a shared INTERNAL_SECRET header so outsiders can't call them: **flag-user** (Python → Node) and **verify-admin** (Python asks Node to confirm a user is really an admin before allowing knowledge-base edits). The student's JWT is also forwarded so Node knows *who* is acting.

4. The AI Engine — Model Choices & Why

A deliberate design decision runs through the whole AI stack: **everything runs locally, with no cloud AI calls**. No student data leaves our server, there is zero recurring API cost, and the system works offline. Here is each model and *why* we chose it.

Component	What we use	Why this choice
Language Model (the “brain” that writes answers)	Llama 3.2 : 3B via Ollama (runs locally)	Free & open; runs on a normal laptop CPU; keeps student data private; no internet or API key needed. The 3B size is the sweet spot between quality and speed on CPU.
Embedding model (turns text into meaning-vectors)	all-MiniLM-L6-v2 (Sentence-Transformers)	Tiny (~27 MB) and fast, yet strong at semantic similarity. Outputs a 384-number vector per text. Industry-standard baseline for retrieval.
Vector database (stores & searches the knowledge)	ChromaDB (persistent, local)	Simple, file-based, no server to run; built for exactly this — fast nearest-neighbour search over embeddings.
Speech-to-text (voice input)	Whisper “base” via faster-whisper	Runs locally on CPU (int8); accurate; multilingual (handles Urdu speech). Replaced a paid cloud API (AssemblyAI).

Why local Llama instead of ChatGPT / GPT-4?

- **Privacy** — students may share names, IDs, and complaints. With a local model, that data never leaves our machine.
- **Cost** — a cloud API charges per request; a university chatbot could get thousands of questions. Local = zero marginal cost.
- **Control & offline** — no rate limits, no API outage, no key to leak. The demo works even with no internet.
- **It is enough for the task** — RAG does the heavy lifting (it supplies the facts); the LLM only has to phrase the retrieved facts well, which a 3B model does comfortably.

Honest engineering note (good to say out loud)

“We kept a **cloud fallback** in the code (Gemini / Groq) behind a single config flag, so if we ever need a bigger model we can switch in one line — but by default the project runs 100% locally on Llama 3.2:3B.” This shows you thought about trade-offs, not that you were limited.

5. What is RAG? (The Heart of the Project)

RAG = Retrieval-Augmented Generation. It is the single most important concept to explain well, because it is what makes our chatbot trustworthy instead of a guessing machine.

The problem with a plain LLM

- It does **not know MAJU-specific facts** (fee figures, exact deadlines, faculty emails).
- It can **hallucinate** — confidently invent a wrong answer.
- Its knowledge is **frozen** at training time and cannot be updated.

The RAG idea — an open-book exam

Instead of asking the model to answer from memory (a closed-book exam, where it might bluff), we give it the relevant pages first and say “answer *using these*.” That is an open-book exam. Concretely: when a student asks a question, we first **retrieve** the most relevant chunks of MAJU website text, then ask the LLM to **generate** an answer grounded only in those chunks.

1. RETRIEVE	2. AUGMENT	3. GENERATE
Find the MAJU text chunks most relevant to the question (semantic + keyword search)	Insert those chunks into the prompt as trusted context	LLM writes a natural answer using only that context — no inventing

Why this matters for us

- **Accuracy** — answers are tied to the real MAJU website, not the model's imagination.
- **Up-to-date & editable** — the admin can add/edit/delete knowledge chunks live; no retraining needed.
- **Anti-hallucination** — our prompt explicitly says: if the context doesn't contain the answer, say “I don't have that information” rather than guess.

If they ask “so is this just ChatGPT?”

“No. ChatGPT answers from its frozen general memory. Our system **searches MAJU's own data** for every question and forces the answer to come from that. The intelligence is in the **retrieval + grounding**, not just the language model — and we built that retrieval pipeline ourselves.”

6. Technology Stack & Libraries — What & Why

Every dependency below is actually used in the project. Know the **bold** ones cold — those are the ones examiners point at.

AI Service — Python (python/)

Package	Role	Why we use it
fastapi + uvicorn	Web API for the AI service	Modern, fast, auto-validating Python web framework; serves /ask, /transcribe, /chunks
chromadb	Vector database	Stores 4,763 embeddings; does nearest-neighbour semantic search locally
sentence-transformers	Embeddings (all-MiniLM-L6-v2)	Converts text ↔ 384-dim vectors so we can search by meaning
ollama	Local LLM client	Talks to the local Llama 3.2:3B daemon to generate answers
faster-whisper	Speech-to-text	Local voice → text (CPU, int8); powers voice questions
langdetect	Language detection	Base classifier; we add a Roman-Urdu lexicon on top
beautifulsoup4 + lxml	Web scraping / HTML parsing	Extracts clean text from the MAJU website pages
requests	HTTP client	Fetches web pages; calls Node's internal webhook
pypdf / python-docx	Document parsing	Admin can upload PDF/DOCX to add knowledge
langchain-text-splitters	Chunking helper	RecursiveCharacterTextSplitter for uploaded documents
pyjwt	Auth	Verifies the JWT Node issued before allowing admin edits
pydantic	Validation	Validates API request/response shapes

Backend — Node.js (server/)

Package	Role	Why we use it
express	Web/API framework	Routes, middleware, REST API for the whole app
mongoose	MongoDB ODM	Schemas & models for users, issues, chats, logs
jsonwebtoken	Authentication	Issues & verifies JWT login tokens
bcryptjs	Password security	Hashes passwords (never stored in plain text)
nodemailer	Email	Gmail SMTP — OTP, notifications, staff credentials
multer	File uploads	Issue attachments (PDF/image) & profile avatars
express-rate-limit	Abuse protection	Throttles repeated requests
cors / dotenv	Config & access	Cross-origin rules; environment secrets
node-fetch / form-data	Service calls	Forwards chat & audio to the Python service

Frontend — React (client/)

Package	Role	Why we use it
react 19 + react-dom	UI library	Component-based, fast student/staff/admin interfaces

Package	Role	Why we use it
@reduxjs/toolkit + react-redux	State management	Predictable global state; async thunks for API calls
tailwindcss + vite	Styling & build	Fast, consistent UI; instant dev server & optimized build
react-router-dom	Routing	Role-based protected routes (student / staff / admin)
axios	HTTP client	Two instances: one for Node, one for the Python service
recharts	Charts	Dashboard trend & analytics graphs
lucide-react	Icons	Clean, consistent icon set
react-hot-toast	Notifications	Non-blocking toast messages
react-markdown	Rendering	Renders the chatbot's formatted answers

7. The End-to-End Pipeline / Flow

There are two pipelines: the **build-time** pipeline that prepares the knowledge (done once / when data changes), and the **live query** pipeline that runs on every question. This is the section examiners love — be able to walk through both.

A. Build-time pipeline — preparing the knowledge (offline)

- 1 **Scrape** — read the MAJU sitemap (jinnah.edu) and crawl all useful pages (16 parallel workers, junk/listing pages filtered out).
- 2 **Clean** — strip nav bars, footers, scripts, ads; keep only real content text.
- 3 **Chunk** — split each page into self-contained topic blocks (heading + its paragraphs). Target ~900 chars, never cut a paragraph in half. Result: **4,763 chunks** from **1,210 pages**.
- 4 **Embed** — convert each chunk to a 384-number vector with all-MiniLM-L6-v2.
- 5 **Store** — save the vectors + text in ChromaDB (the `university_chunks` collection).

B. Live query pipeline — what happens on every question

- 1 **Language detect** — classify the message as English / Roman-Urdu / mixed (langdetect + a Roman-Urdu function-word lexicon for SMS-style text like “fees kitni hai”).
- 2 **Moderation** — scan for English + Roman-Urdu profanity. If abusive: refuse politely, flag the student, and alert the admin (via the internal webhook). No answer is generated.
- 3 **Build retrieval query** — for follow-ups (“or fee?”), prepend the previous question so the search stays on-topic.
- 4 **Hybrid retrieval** — run two searches and merge: **semantic** (vector similarity, catches paraphrases) + **keyword** (TF-IDF, catches exact tokens like course codes, emails, fees). Take the top 8 chunks.
- 5 **Build the prompt** — insert the chunks as context, plus rules: answer only from context, don't invent, reply in the detected language, never reveal the AI model.
- 6 **Generate** — send to Llama 3.2:3B via Ollama (temperature 0.3 for focused answers).
- 7 **Clean** — strip filler preambles (“Sure!”, “Based on the context...”) and any citation markers.
- 8 **Language guards (Roman-Urdu)** — if the model slips into Hindi/Devanagari script, retry once then strip it; swap leaked Hindi words for Urdu equivalents (e.g. *sampark* → *raabta*).
- 9 **Return** — send the final answer back to the chat, with conversation history kept for follow-ups.

One-sentence version of the flow

“Question → detect language → check for abuse → search MAJU knowledge (semantic + keyword) → put the best 8 chunks into a prompt → local Llama writes the answer → clean it & fix the language → reply.”

Why hybrid retrieval (a likely deep question)

Semantic search alone is great for meaning (“cost of CS” finds “BSCS fee structure”) but it can **under-weight exact tokens** — a course code like “CS-3413”, a phone number, or a specific fee figure. The keyword pass catches those. We merge both, de-duplicate, and rank — getting the best of meaning *and* exactness.

8. Key Features — What Each Module Does

AI Chatbot

- RAG answers grounded in the MAJU website; English / Roman-Urdu / mixed.
- **Voice input** — record a question; Whisper transcribes it locally.
- **Conversation memory** — resolves follow-ups (“and its fee?”).
- **Profanity filter** — abusive messages are blocked, the student is flagged, and the admin is notified.

Issue Tracker (Complaint / Help system)

- Student submits an issue to a department (SFC, IT, HOD) with up to 3 attachments.
- Status workflow: Pending → In Progress → Resolved → Closed (or Rejected with a reason).
- **Live sync every 20s**; every change fires an email + in-app bell notification.
- Staff see **only their department's** issues; can assign/re-assign and coordinate so two staff don't clash (concurrency guard).

Role-Based Portals (RBAC)

- **Student**: chat, issues, profile.
- **Staff**: department dashboard (trends, avg first-response time) + department inbox.
- **Admin**: full dashboard, user management (block/flag), staff creation with auto-generated credentials, departments, cross-department analytics, audit/login/chat logs.
- Enforced by JWT + a `requireRole` middleware; blocked users are force-logged-out.

Live Knowledge-Base Editor (a standout feature)

From the Admin panel, the admin can **list, search, add, edit, and delete** the chatbot's knowledge chunks — or upload a PDF/DOCX/TXT that is automatically chunked and embedded. The chatbot's brain is **editable from the UI**, with no code change and no retraining. Edits re-embed instantly.

Security & audit (good to mention unprompted)

Passwords are bcrypt-hashed; logins (success *and* failure) are logged; every admin write is recorded in an audit log with secrets redacted; account email changes send a security alert to the old address. These show the project takes real-world security seriously.

9. How to Present — Start-to-Finish Script

Evaluation format: a panel of teachers comes to your station and says “tell us about your project.” They watch a short demo and ask questions. Here is exactly how to run it.

Before they arrive — setup checklist

- **Start all three services:** Node (3000), Python (8000), React (5173). Confirm each is up.
- **Warm the chatbot:** send one question so Llama is loaded in RAM (first answer is slow ~30–60s cold; after that it's fast). **Never demo cold.**
- Log in tabs ready: one **Student**, one **Staff**, one **Admin** (so you don't fumble logins live).
- Have seed data: at least one department, one staff account, and 1–2 existing issues.
- Internet optional — but if Wi-Fi is shaky, remember the AI is local and still works.

Step 1 — Opening (30–45 seconds)

Smile, introduce the team, then give the **30-second pitch** from Section 1. End the opening with the scope line: “In FYP-1 we built the core platform, the full AI chatbot, and the three role-based portals; jobs, scholarships, events and the mobile app are planned for FYP-2.”

Step 2 — Demo in story order (the spine of your presentation)

- 1 Guest landing page** → “this is what a visitor sees.” (5 sec)
- 2 Student: the chatbot** → ask a real MAJU question in English (“What programs does MAJU offer?”). Then ask the **same kind in Roman Urdu** (“BSCS ki fee kitni hai?”). Then a **voice question**. This is your showpiece — spend the most time here.
- 3 Show grounding** → ask something off-topic (“what's the weather?”) to show it politely declines & stays on MAJU — proving it doesn't hallucinate.
- 4 Student: raise an issue** → submit a complaint to a department with an attachment.
- 5 Staff portal** → switch tab, show the issue arrived in that department's inbox, reply + change status.
- 6 Back to student** → show the notification (bell + the status change).
- 7 Admin panel** → dashboard stats, then the **live knowledge-base editor** (add a chunk, ask the bot about it — it now knows). Show logs & a flagged user if time allows.

Step 3 — While you click, narrate the “why”

Don't just click — say what's happening underneath: “right now this question is being embedded, searched against 4,763 MAJU chunks, and answered by a **local** Llama model — no data leaves our server.” That one sentence signals depth.

Step 4 — Closing (20–30 seconds)

“To summarize: UniAssist already gives MAJU students a single, AI-powered place to get answers and raise issues, with full admin control — all running on a private, local AI stack. In FYP-2 we'll add the job, scholarship and events modules and a mobile app on top of this foundation. Thank you — we're happy to take questions.”

Do's and Don'ts

DO	DON'T
Warm the model before the demo	Demo the chatbot cold (long awkward wait)
Say “local, private, no cloud cost” often	Claim it's smarter than ChatGPT
Admit FYP-2 items clearly	Pretend job/scholarship modules are done
Let each member speak on their part	One person talks the whole time
Keep answers short, then offer detail	Ramble or guess if unsure — say you'll check

Suggested speaker split (adjust as you like)

Member	Owns / presents
Syed Ibrahim	Opening pitch + the AI chatbot & RAG pipeline (the core)
Mubashir Riaz	Issue tracker + staff portal demo
Adnan Kamal	Admin panel + knowledge-base editor + security/RBAC
Abdul Rehman	Architecture, tech stack & FYP-2 roadmap + closing

10. Possible Questions & Answers

Grouped by what examiners typically probe. Answer in 2–3 sentences, then stop — let them ask for more.

A. Conceptual / “what is”

Q. What is your project, in one line?

A. A centralized AI student portal for MAJU: a Retrieval-Augmented chatbot plus role-based Student, Staff and Admin portals for getting information and raising issues.

Q. What is RAG and why did you use it?

A. Retrieval-Augmented Generation. We first retrieve relevant MAJU website text, then the LLM answers using only that. It keeps answers accurate and grounded, prevents hallucination, and lets us update knowledge without retraining.

Q. What does the chatbot actually run on?

A. A local Llama 3.2 3B model served by Ollama, with an all-MiniLM-L6-v2 embedding model and a ChromaDB vector store of 4,763 chunks scraped from the MAJU website. Everything runs locally.

Q. Is this just ChatGPT with extra steps?

A. No. ChatGPT answers from frozen general memory. Ours searches MAJU's own data for every question and forces the answer to come from that. We built the scraping, chunking, embedding and hybrid-retrieval pipeline ourselves.

B. Why these choices (design rationale)

Q. Why a local model and not GPT-4 / a cloud API?

A. Privacy (student data never leaves our server), zero per-request cost, offline reliability, and no API keys to leak. RAG supplies the facts, so a 3B model is enough to phrase them well.

Q. Why all-MiniLM-L6-v2 for embeddings?

A. It's small (~27 MB), fast on CPU, and a strong, widely-used baseline for semantic similarity. It outputs 384-dim vectors — a good quality/speed trade-off for our corpus size.

Q. Why ChromaDB and not a normal SQL database?

A. A normal DB searches by exact keywords. ChromaDB searches by meaning using vector similarity, so “tuition cost” finds “fee structure”. It's also local, persistent and zero-config.

Q. Why a separate Python service instead of doing AI in Node?

A. The AI/ML ecosystem (embeddings, ChromaDB, Whisper, Ollama) is Python-first. Isolating it as a FastAPI microservice keeps Node clean and lets us scale or swap either side independently.

Q. Why MongoDB?

A. Our data (users, issues with nested replies/attachments, chats, logs) is document-shaped and evolves quickly. MongoDB's flexible schema fits that better than rigid tables for an evolving FYP.

C. Technical depth

Q. How do you turn text into something searchable?

A. Each chunk is passed through the embedding model to get a 384-number vector that captures its meaning. The question is embedded the same way, and ChromaDB returns the chunks whose vectors are closest.

Q. What is your retrieval strategy?

A. Hybrid: semantic (vector similarity) for paraphrases plus a keyword/TF-IDF pass for exact tokens like course codes, emails and fees. We merge, de-duplicate, and take the top 8 chunks.

Q. How do you do chunking?

A. Structurally, not by raw character count — one chunk is a heading plus its paragraphs (~900 chars target, 1600 max, never cutting a paragraph in half). This keeps each chunk a self-contained topic.

Q. How do you prevent hallucination?

A. Three ways: the prompt forbids using outside knowledge and tells it to say 'I don't have that info' if the context is silent; retrieval grounds it in real data; and we keep temperature low (0.3) for focused answers.

Q. How does Roman-Urdu support work?

A. A language detector (langdetect + a Roman-Urdu function-word lexicon for SMS-style text) tags the message; the prompt then instructs the model to reply in that language. We also guard the output — strip any Hindi/Devanagari script and swap leaked Hindi words for Urdu ones.

Q. How does voice input work?

A. The browser records audio, Node forwards it to the Python service, and faster-whisper transcribes it locally to text, which then goes through the same RAG pipeline.

Q. How does the admin update the chatbot's knowledge?

A. From the Admin panel they can add/edit/delete chunks or upload a PDF/DOCX/TXT, which is auto-chunked and embedded. Edits re-embed instantly — no retraining, no code change.

Q. What are temperature and top-k?

A. Temperature (0.3) controls randomness — low means focused, factual answers. Top-k (8) is how many knowledge chunks we feed the model per question.

D. Scope & project management

Q. What did you complete in FYP-1 vs FYP-2?

A. FYP-1: the full AI chatbot, the issue-tracking system, and all three role-based portals end to end. FYP-2: job portal, scholarship matching, events, a unified profile, and the mobile app.

Q. Why are the Jobs/Scholarship/Events pages empty?

A. They're intentional placeholders so the routing and navigation are ready; the modules themselves are FYP-2 work that builds on this foundation.

Q. How is real-time done?

A. Currently 20-second polling on issue lists, dashboards and the notification bell (it pauses when the tab is hidden). For FYP-2 / scale we'd move to WebSockets.

E. Security & reliability

Q. How is authentication handled?

A. JWT-based: login returns a signed token; a middleware verifies it on protected routes and a requireRole check enforces student/staff/admin access. Passwords are bcrypt-hashed and blocked users are force-logged-out.

Q. How do the Node and Python services trust each other?

A. Internal endpoints use a shared INTERNAL_SECRET header, and for admin actions Python forwards the user's JWT so Node can confirm they're really an admin before allowing knowledge-base edits.

Q. What if the website content changes?

A. We re-run the scrape-clean-chunk-embed pipeline to refresh the knowledge, or the admin edits chunks directly from the UI for small changes.

11. Anticipated Objections & Confident Responses

These are the “gotcha” questions meant to test if you understand limits. Stay calm, agree where fair, then reframe to a strength or a clear FYP-2 plan.

■ “A 3B model is small and weak.”

True, it's small — but that's by design. In RAG the model doesn't need world knowledge; it just phrases facts we already retrieved. A 3B model does that well, runs on a laptop, and keeps everything local and free. We also kept a one-line switch to a larger cloud model if ever needed.

■ “Why not just use ChatGPT / GPT-4?”

For a university chatbot, privacy and cost matter: student data shouldn't go to a third party, and per-request cloud billing doesn't scale to thousands of questions. Local also means it works offline with no API outage.

■ “This is just a wrapper around an LLM.”

The LLM is one component. We built the scraping, cleaning, structural chunking, embedding, a hybrid semantic+keyword retriever, the moderation layer, the multilingual detection and output guards, and a live editable knowledge base. That pipeline is the engineering.

■ “Scraping the website — is that reliable / allowed?”

We scrape MAJU's own public website for a MAJU project, and only clean text. If the site changes we re-run the pipeline; and the admin can edit knowledge directly, so we're not fully dependent on the scrape.

■ “How accurate is Roman Urdu, really?”

We handle it in layers — detection tuned for SMS-style spellings, a prompt that forces the right language and vocabulary, and post-processing that removes wrong scripts and swaps Hindi words for Urdu. It's not perfect, but it's robust for common student phrasing, and improving it is ongoing.

■ “Polling every 20 seconds won't scale.”

Correct for large scale — we chose polling for FYP-1 simplicity and reliability. Moving to WebSockets is a planned FYP-2 improvement; for our current user volume polling is perfectly adequate.

■ “There are no automated tests.”

We verified features by exercising the full flows manually and documented a verification checklist. A formal test suite is a fair next step we'd add in FYP-2 as the codebase stabilises.

■ “What stops abuse or offensive use of the bot?”

A moderation layer scans English and Roman-Urdu profanity before any answer; offensive messages are blocked, the student is flagged on their record, and the admin is notified live.

■ “What if the model invents a fee or deadline?”

The prompt explicitly bans inventing facts and tells it to fall back to ‘I don't have that information’. Because answers are grounded in retrieved MAJU text, there's no need for it to guess.

12. Cheat Sheet — Memorise These

Numbers to know cold

Thing	Value
LLM	Llama 3.2 : 3B (local, via Ollama)
Embedding model	all-MiniLM-L6-v2 — 384 dimensions, ~27 MB
Vector DB	ChromaDB — collection 'university_chunks'
Knowledge size	4,763 chunks from 1,210 MAJU pages
Retrieval	Hybrid (semantic + keyword), top-k = 8
Chunk size	~900 chars target, 1600 max, 120 min
LLM settings	temperature 0.3, max ~1000 tokens
Speech-to-text	Whisper 'base' via faster-whisper (local, CPU)
Roles	Student, Staff, Admin (JWT + requireRole)
Departments	SFC, IT, HOD
Live sync	20-second polling (pauses on hidden tab)
Stack	React 19 / Node+Express / FastAPI + Python; MongoDB + ChromaDB

One-liners (say these verbatim)

- “Everything runs **locally** — no student data leaves our server, zero cloud cost.”
- “RAG means the answer comes from **MAJU's own data**, not the model's imagination.”
- “The admin can **edit the chatbot's brain from the UI** — no retraining.”
- “It understands **Roman Urdu and voice**, the way real students talk.”
- “FYP-1 = core platform + AI + portals. FYP-2 = jobs, scholarships, events, mobile.”

If you get stuck on a question

Say: “That's a good question — our current design handles X this way, and we've noted Y as a planned improvement.” Never guess a fact; offer to show it in the code or demo instead.

Prepared from the verified UniAssist codebase — python/, server/, client/. Good luck, team. You built something real; present it with confidence.